

CSCI1409 Deep Learning

Week 1 — Introduction + Mathematical Foundations

Abirov Aidyn



KAZAKH-BRITISH
TECHNICAL
UNIVERSITY

Data Scientist / Senior Lecturer

- Current company: Wildberries & Russ
- Experience: 5.5 years of Experience in Industry
- Degree: Masters in Control Science and Control Engineering
- University: Nanjing University of Science and Technology
- Conferences: DataFest 2024, Stachka 2025, Kolesa Conf 2025
- Conference paper: 2023 2nd International Symposium on Computing and Artificial Intelligence



KAZAKH-BRITISH
TECHNICAL
UNIVERSITY

Wildberries & Russ

- Solved a business-critical repacking control use case for Wildberries warehouses; the solution is deployed across 10 large warehouses and operates on 1000+ cameras



Fermata Agro Tech

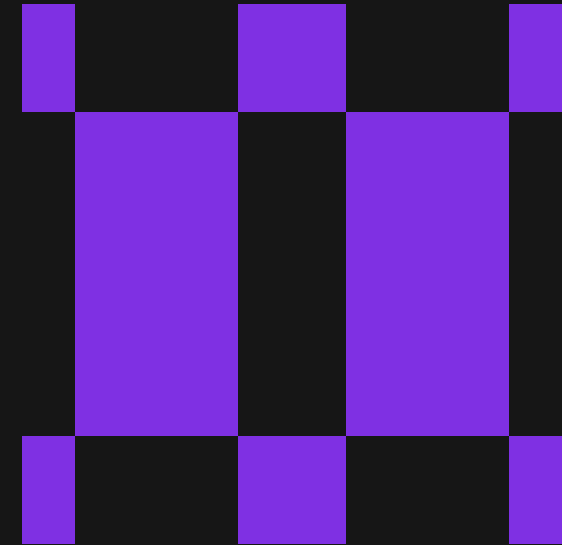
- Solved a tomato disease classification problem from cameras. Computer Vision models were able to detect 8 different diseases on the early stage

FERMATA

Lectures, Talks & Mentorship



- DataFest 2024, Almaty, Kazakhstan
- Online Webinar 2025, Moscow, Russia
- Hackathon mentor 2025, Minsk, Belarus
- Kolesa Conf 2025, Almaty, Kazakhstan
- Nazarbayev University Guest Lecture 2025, Astana, Kazakhstan
- KBTU Guest Lecture 2025, Almaty, Kazakhstan
- KBTU Member of Industrial Committee



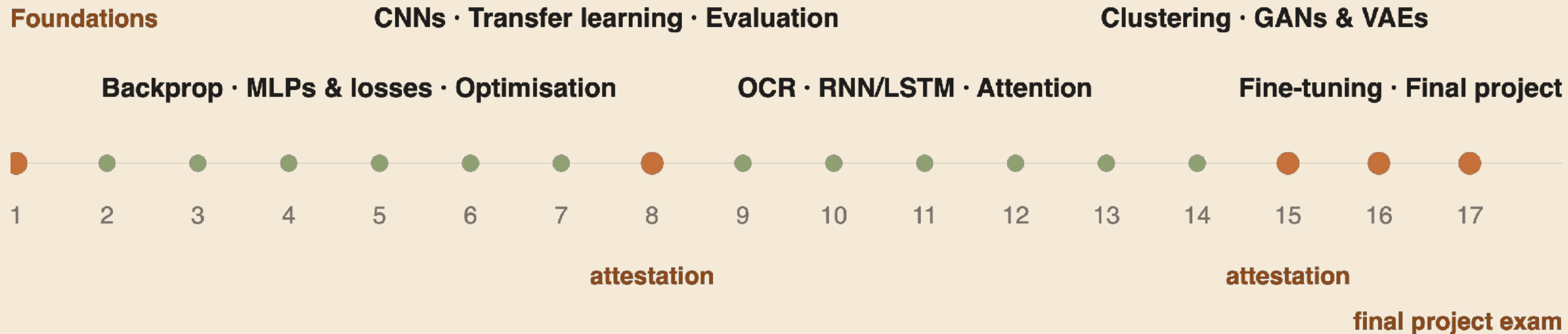
Important Contacts

MS Teams code: smigr55

Email: a.abirov@kbtu.kz

LinkedIn: <https://www.linkedin.com/in/aidynabirov/>

Where we're going



Linear & logistic regression → MLPs → CNNs → OCR → RNN/LSTM → Attention
15 weeks. Everything builds on today.

Week	Topic	Lectures (hours)	Practice (hours)	SIS defense
1	Introduction to Deep Learning + Mathematical Foundations	2	1	
2	Perceptron + Derivatives, Gradients & Backpropagation	2	1	
3	Multi-Layer Perceptrons (MLPs) + Loss Functions	2	1	
4	Optimization Methods for Deep Learning	2	1	*
5	Convolutional Neural Networks (CNNs): Foundations	2	1	
6	CNN Applications + Transfer Learning + Pretrained Models	2	1	
7	Model Evaluation Metrics + Regularization	2	1	*
8	Midterm	2	1	

Week	Topic	Lectures (hours)	Practice (hours)	SIS defense
9	Optical Character Recognition (OCR)	2	1	
10	RNNs and LSTMs	2	1	
11	Attention Mechanism + Transformers	2	1	*
12	Unsupervised Learning: Clustering + Autoencoders	2	1	
13	Generative Models: GANs and VAEs	2	1	
14	Modern Deep Learning: Fine-tuning + Revision	2	1	*
15	Endterm	2	1	
16–17	Final Exam			

How you are graded

COMPONENT	POINTS
Attendance / participation	4
Practice (TSIS)	8
SIS labs	16
Midterm + Endterm	20
Quizzes	12
Final exam	40
Total	100

When you are graded

- TSIS practice: weeks 2, 3, 5, 6, 9, 10, 12, 13 — 1 point each
- SIS labs: weeks 4, 7, 11, 14 — 4 points each
- Midterm week 8, Endterm week 15 — 10 points each
- Final project exam, weeks 16–17 — 40 points, groups of up to 4

Three ways to fail automatically

- Miss more than 30% of lessons → **F**
- Score 29.4 or less across the two attestations → not admitted to the final → **F**
- Score 9.4 or less on the final → **F**, regardless of semester points

AI policy: Level C (AI-Assisted)

Permitted: ideas, explanations, writing help, debugging, feedback

Required: you author the solution; you understand and verify everything you submit

Plagiarism, duplication and falsified data: never, under any circumstances

In lab defenses I will point at a line of your code and ask you to explain it

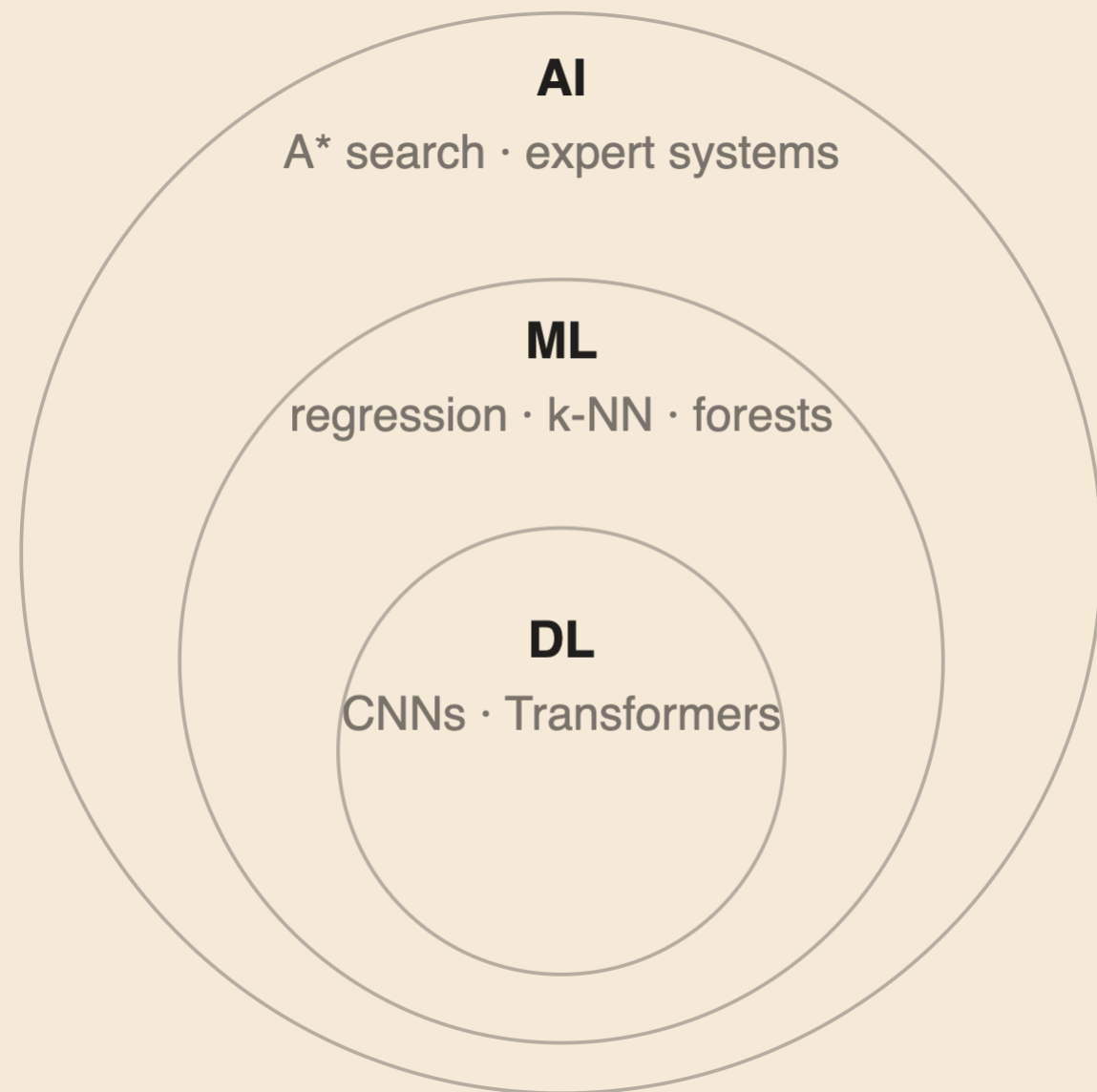
Tools

Today: Python 3.10+, NumPy, Matplotlib — nothing else

From Week 2: PyTorch (CPU build is fine)

Install before next week. Week 2's TSIS assumes `import torch` works.

AI $\supset$ ML $\supset$ DL



AI — any system doing something we'd call intelligent

ML — behaviour induced *from data*, not programmed by hand

DL — compositions of parameterised layers that learn the *representation*

Where deep learning wins, and where it doesn't

USE DL

Images, audio, text, video

Abundant labels, or pretraining to transfer from

Nobody can write down good features

—

DON'T USE DL

Small tabular data (trees win)

Decisions you must justify to a regulator

A known exact algorithm already exists

Tight compute or latency budgets; causal questions

Three generations of a spam filter

Rules — `if "free money" in subject. Breaks on fr€e m0ney.`

Classical ML — you pick the features, the weights adapt. *This is weeks 1–3.*

Deep learning — the model learns its own representation from raw tokens.

The artificial neuron

$$\mathbf{z} = \mathbf{w}^\top \mathbf{x} + \mathbf{b} = \sum_{i=1}^n w_i x_i + \mathbf{b}, \quad \mathbf{a} = \varphi(\mathbf{z})$$

$\mathbf{z}$ = pre-activation · φ = activation function · $\mathbf{a}$ = activation

Perceptron (Rosenblatt, 1958): $\varphi(\mathbf{z}) = 1[\mathbf{z} > 0]$ — a hyperplane

Layer, then network

$$\text{layer: } \mathbf{a} = \varphi(\mathbf{W}\mathbf{x} + \mathbf{b}), \quad \mathbf{W} \in \mathbb{R}^{m \times n}$$

$$\text{network: } f_{\theta}(\mathbf{x}) = \varphi_L(\mathbf{W}_L \varphi_{L-1}(\cdots \varphi_1(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) \cdots) + \mathbf{b}_L)$$

Row j of $\mathbf{W}$ is neuron j 's weight vector. **One matrix = one layer.**

Learnin = choosing the numbers in θ . Nothing more mystical than that.

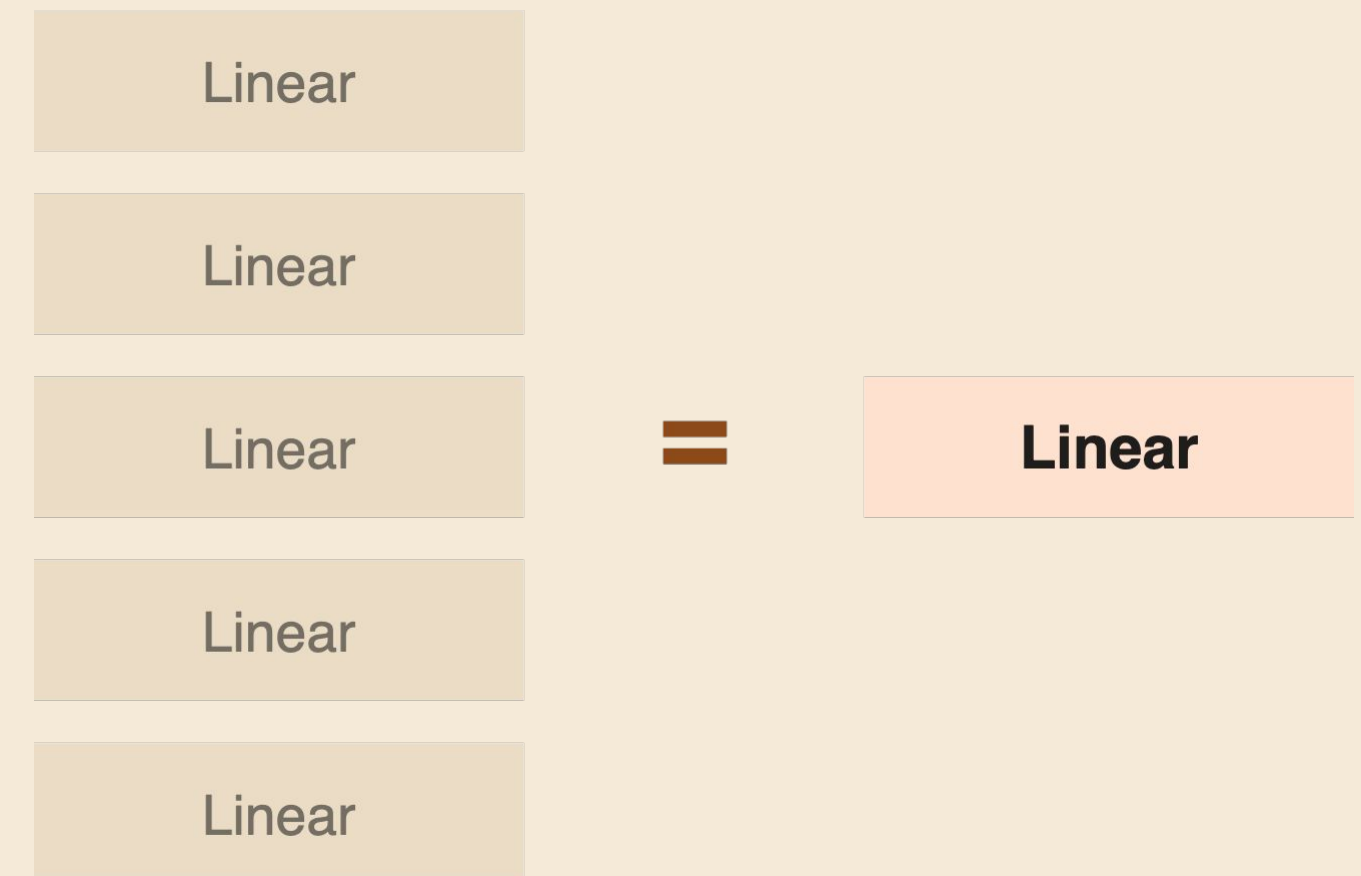
g

Why the nonlinearity is not optional

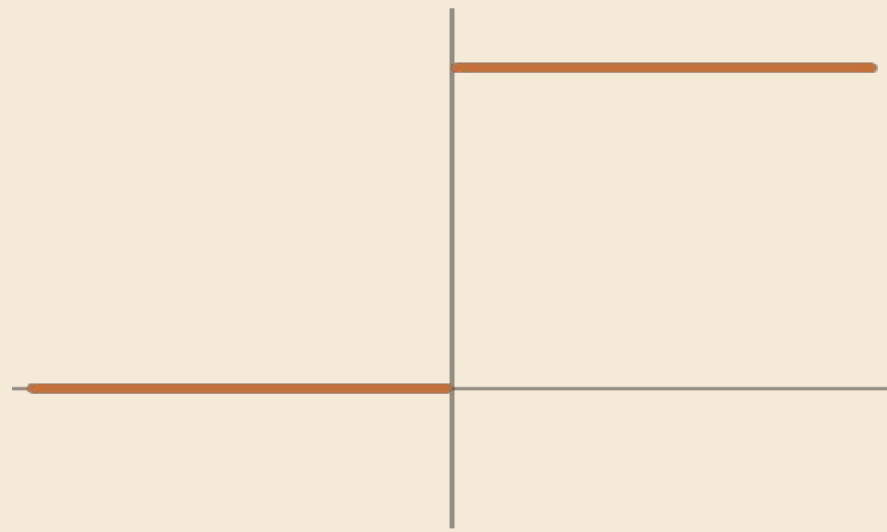
$$h = W_1 x + b_1, \quad y = W_2 h + b_2$$

$$y = \underbrace{(W_2 W_1)}_{W'} x + \underbrace{(W_2 b_1 + b_2)}_{b'}$$

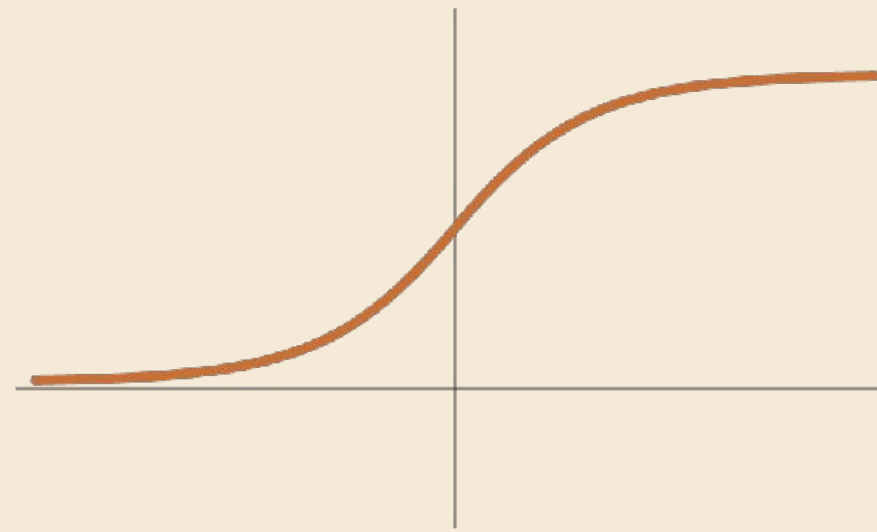
Without ϕ , any depth collapses to depth 1.



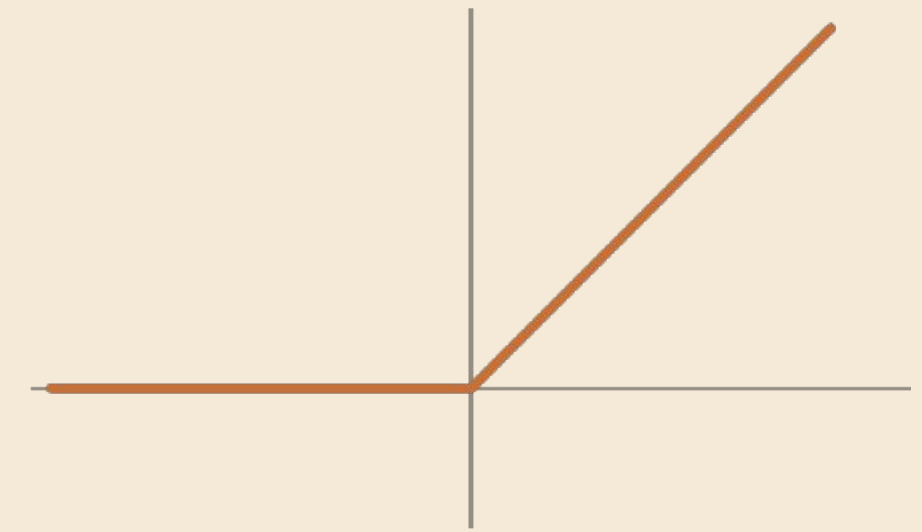
Three activation functions



Step: $\mathbb{1}[z > 0]$



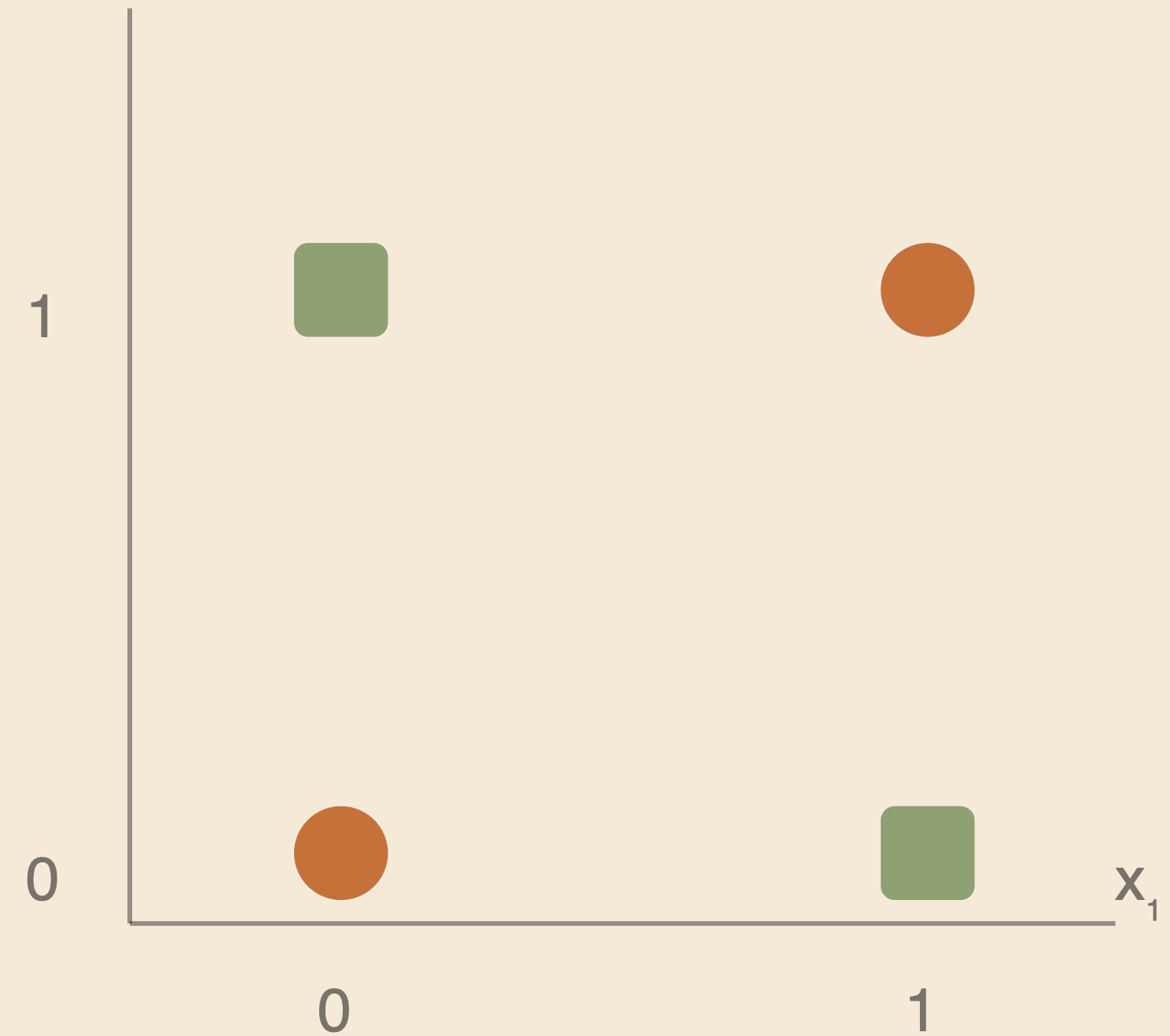
Sigmoid: $\sigma(z) = 1/(1+e^{-z})$



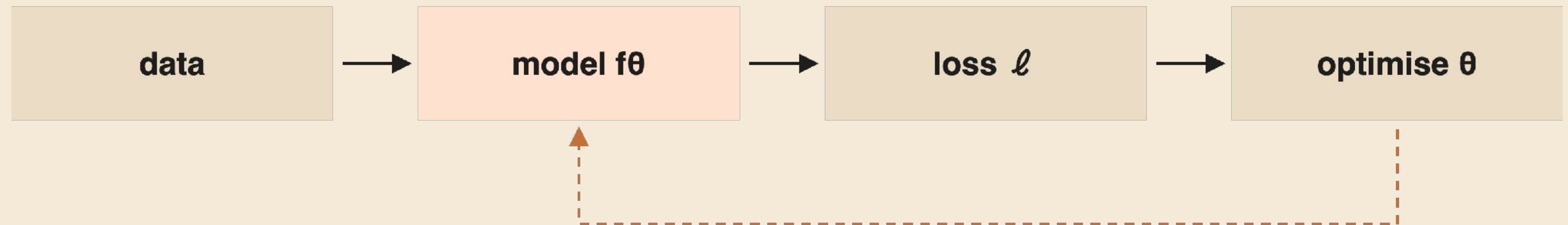
ReLU: $\max(0, z)$

XOR: one line is not enough

x_1	x_2	Y
0	0	0
0	1	1
1	0	1
1	1	0



The supervised skeleton



Everything else in this course is a variation on these four objects.

What we minimise, and what we actually care about

$$\hat{R}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(\mathbf{x}_i), y_i) \quad R(\theta) = \mathbb{E}_{(\mathbf{x}, y) \sim p} [\ell(f_{\theta}(\mathbf{x}), y)]$$

We can compute the left one. We are graded on the right one.

Generalisation gap = $R(\theta) - \hat{R}(\theta)$

train

val

test

opened once

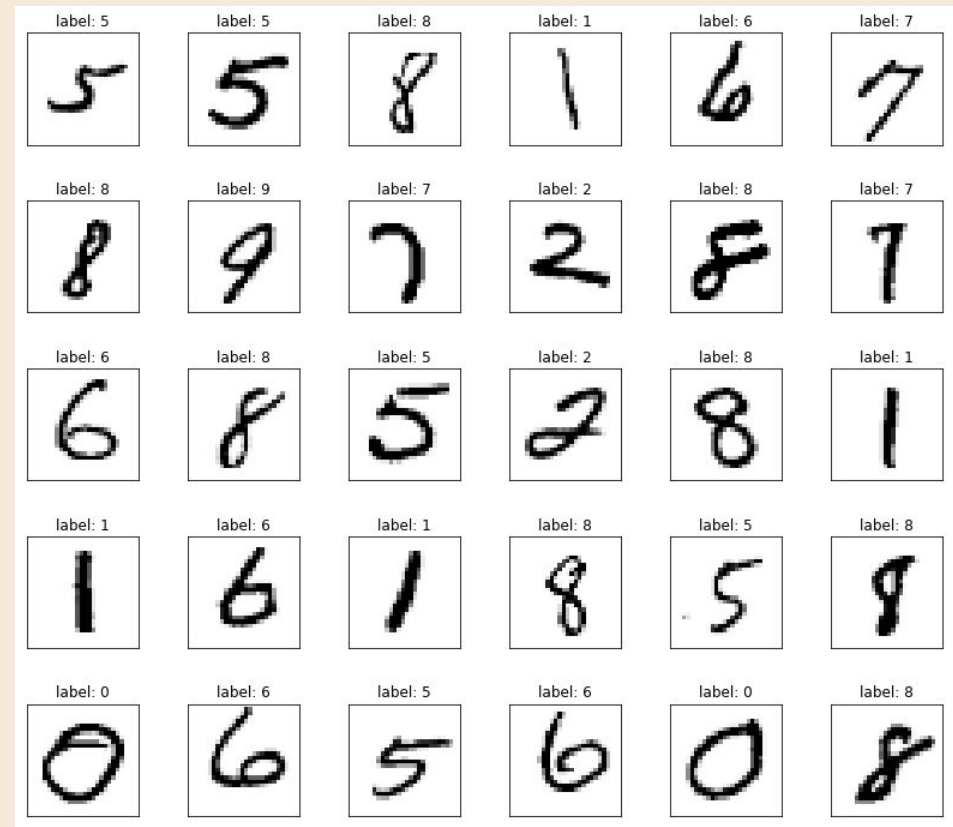
Two task types

Regression — $y \in \mathbb{R}$, loss: mean squared error

Binary — $y \in \{0, 1\}$, sigmoid output, binary cross-entropy
classification

Multi-class — $y \in \{1, \dots, K\}$, softmax output, cross-entropy

100% accuracy is not the target



Which digit is the fifth one?

Why this mattered: reading handwriting

Input was **raw pixels**, not features an engineer designed

Measured on **held-out** data

Architecture encoded a **prior**: nearby pixels relate; a digit is a digit anywhere in the frame

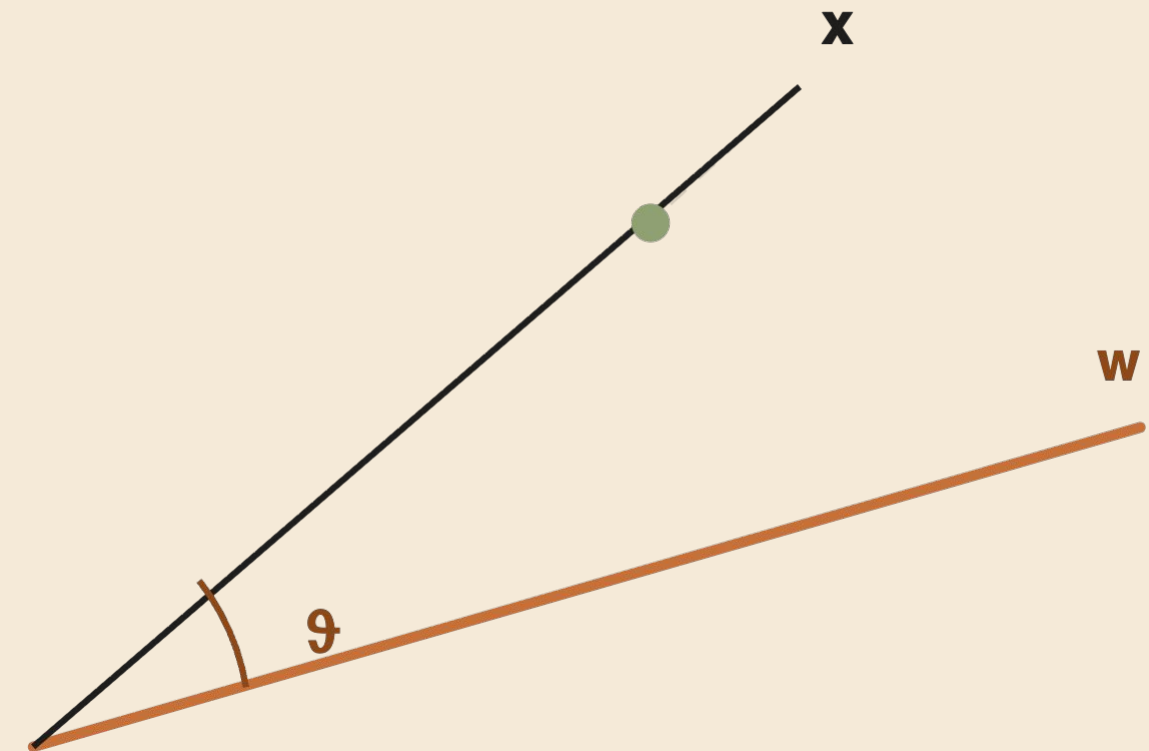
The dataset outlived the system — **MNIST**, which you train on in Week 4

A neuron is a template matcher

$$\mathbf{w}^T \mathbf{x} = \sum_{i=1}^n w_i x_i = \|\mathbf{w}\| \|\mathbf{x}\| \cos \theta$$

The dot product measures **how much of $\mathbf{x}$ points along $\mathbf{w}$**

So a neuron fires when the input resembles its weight vector



The shape ledger

OBJECT

SHAPE

MEANING

x

$(n,)$

one example

X

(N, n)

a batch

W

(m, n)

one layer, m neurons

b

$(m,)$

one bias per neuron

$Z = X @ W.T + b$

(N, m)

pre-activations for the batch

A layer, computed by hand

$$W = \begin{bmatrix} 1 & -1 & 2 \\ 0 & 3 & 1 \end{bmatrix}, \quad x = \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix}, \quad b = \begin{bmatrix} 0.5 \\ -1 \end{bmatrix}$$

$$Wx = \begin{bmatrix} 1 \\ 3 \end{bmatrix} \rightarrow Wx + b = \begin{bmatrix} 1.5 \\ 2.0 \end{bmatrix} \rightarrow \text{ReLU} = \begin{bmatrix} 1.5 \\ 2.0 \end{bmatrix}$$

Broadcasting: the silent bug

$(N, m) + (m,) \rightarrow (N, m)$ — same bias added to every row

$(N, m) + (N, 1) \rightarrow (N, m)$ — same bias added to every column

$(N, 1) + (1, N) \rightarrow (N, N)$ — no error, no warning, wrong answer

The derivative, read usefully

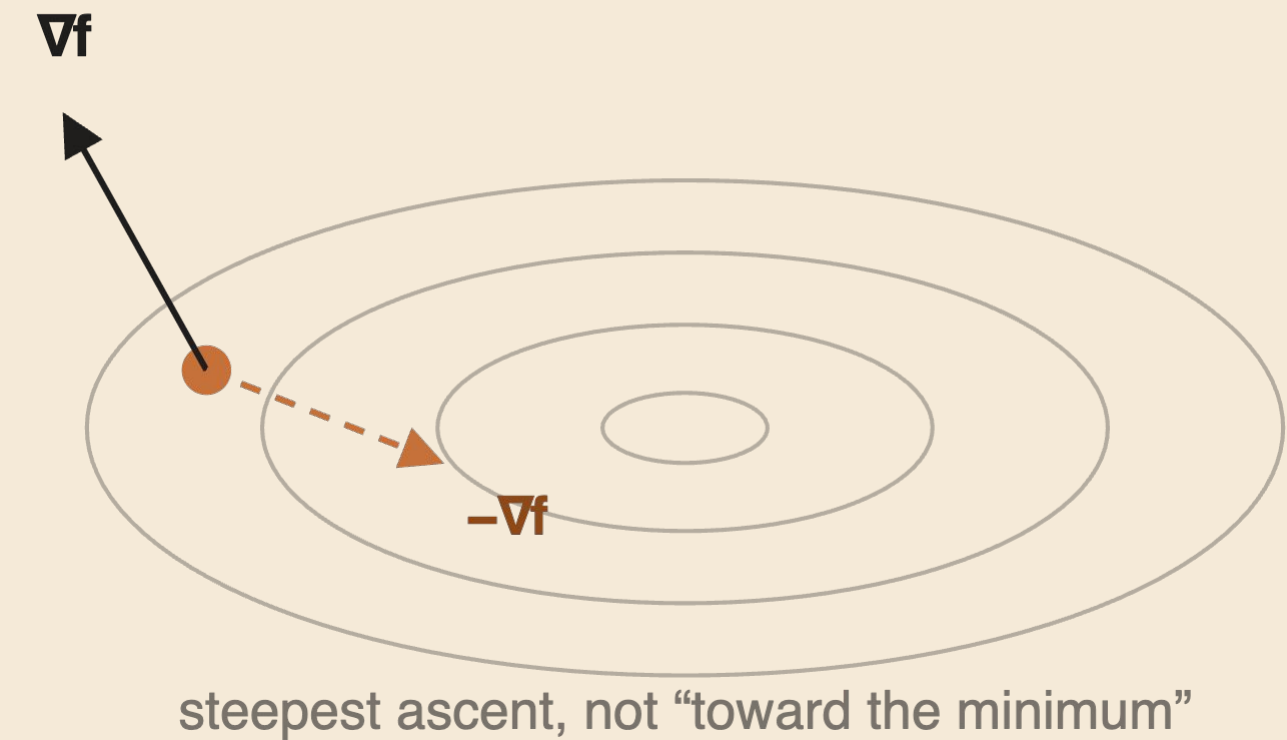
$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \implies f(x+h) \approx f(x) + f'(x)h$$

“If I nudge the input by h , how much does the output move?”

The gradient points uphill

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right]^\top$$

Direction of **steepest ascent** — not “toward the minimum”



The update rule

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \hat{R}(\theta)$$

$\eta > 0$ is the **learning rate**

This single line is what training is. Everything else is detail.

The chain rule

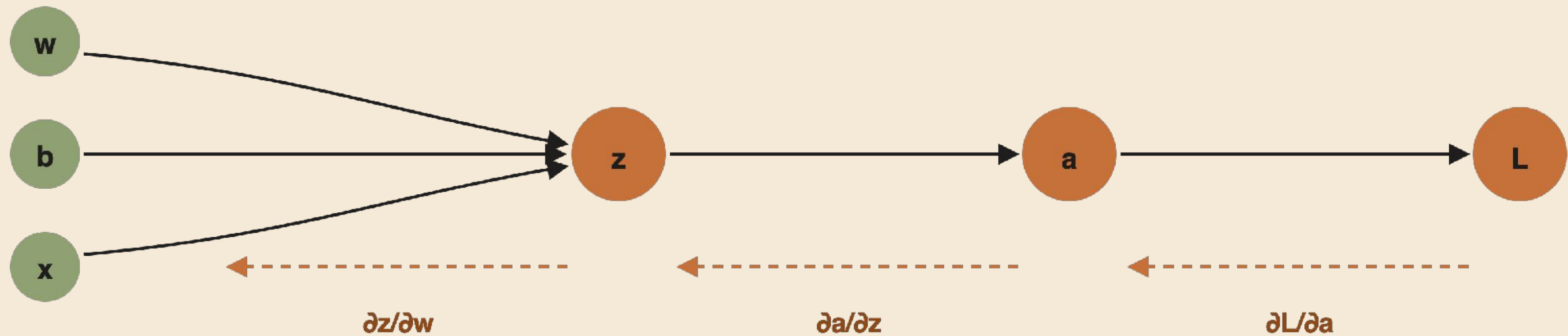
$$u = g(x), L = f(u) \implies \frac{dL}{dx} = \frac{dL}{du} \cdot \frac{du}{dx}$$

several intermediates:

$$\frac{\partial L}{\partial x} = \sum_{j=1}^k \frac{\partial L}{\partial u_j} \cdot \frac{\partial u_j}{\partial x}$$

A network **is** a composition — so this is not a technique we chose.

The computational graph



Forward: compute. Backward: differentiate.

One neuron, differentiated completely

$$z = wx + b, \quad a = \sigma(z), \quad L = (a - y)^2$$

$$\frac{\partial L}{\partial w} = \underbrace{2(a - y)}_{\partial L / \partial a} \cdot \underbrace{a(1 - a)}_{\sigma'(z)} \cdot \underbrace{x}_{\partial z / \partial w}$$

$$x = 2, y = 1, w = 0.3, b = -0.1 \quad z = 0.5 \quad a = 0.622459 \quad L = 0.142537$$

$$\partial L / \partial w = -0.354897 \quad \partial L / \partial b = -0.177448$$

Why a sample average is allowed

$$\mathbb{E}[g(X)] = \sum_x g(x)p(x) \quad \text{or} \quad \int g(x)p(x) dx$$

We want $R(\theta)$, the expected loss — but p is unknown

We have an i.i.d. sample, so we minimise the sample mean instead

That substitution has a name: **empirical risk minimisation**

Where MSE comes from

$$y \mid \mathbf{x} \sim \mathcal{N}(\mathbf{f}_\theta(\mathbf{x}), \sigma^2)$$

$$-\log \prod_i p_\theta(y_i \mid \mathbf{x}_i) = \sum_i \frac{(y_i - \mathbf{f}_\theta(\mathbf{x}_i))^2}{2\sigma^2} + \frac{N}{2} \log(2\pi\sigma^2)$$

Drop the constants → **mean squared error**.

Objective vs metric

OBJECTIVE — TRAINED ON

METRIC — REPORTED

MSE

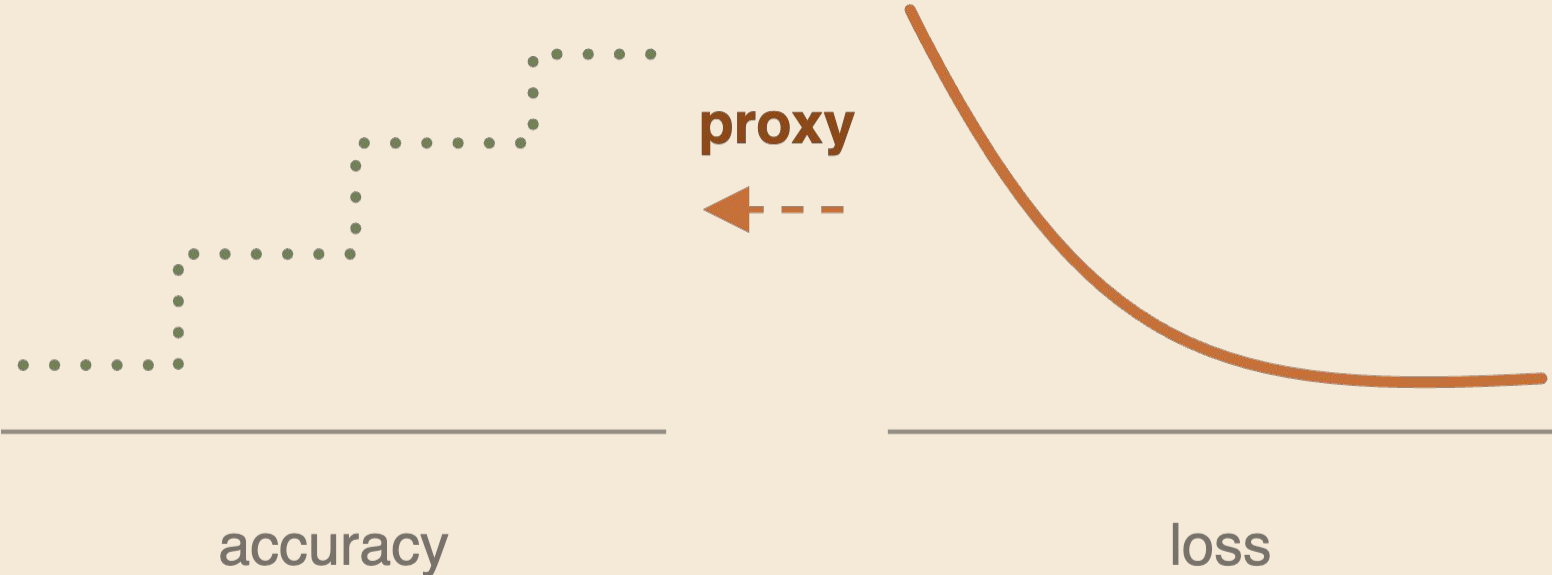
Accuracy

Cross-entropy

Precision, recall, F1

—

ROC-AUC, IoU



The full loop: setup

$$\mathcal{D} = \{(1, 2), (2, 4), (3, 5)\}, \quad \hat{y} = \mathbf{w}\mathbf{x} + \mathbf{b}$$

$$\mathcal{L} = \frac{1}{N} \sum_i (\hat{y}_i - y_i)^2$$

Initialise $\mathbf{w} = 0, \mathbf{b} = 0$. Learning rate $\eta = 0.1$.

All predictions are 0 $\rightarrow$ residuals $-2, -4, -5$

15

the starting loss, $\mathcal{L} = \frac{45}{3}$

Gradients: derive, then substitute

$$\frac{\partial L}{\partial \mathbf{w}} = \frac{2}{N} \sum_i (\hat{y}_i - y_i) \mathbf{x}_i, \quad \frac{\partial L}{\partial \mathbf{b}} = \frac{2}{N} \sum_i (\hat{y}_i - y_i)$$

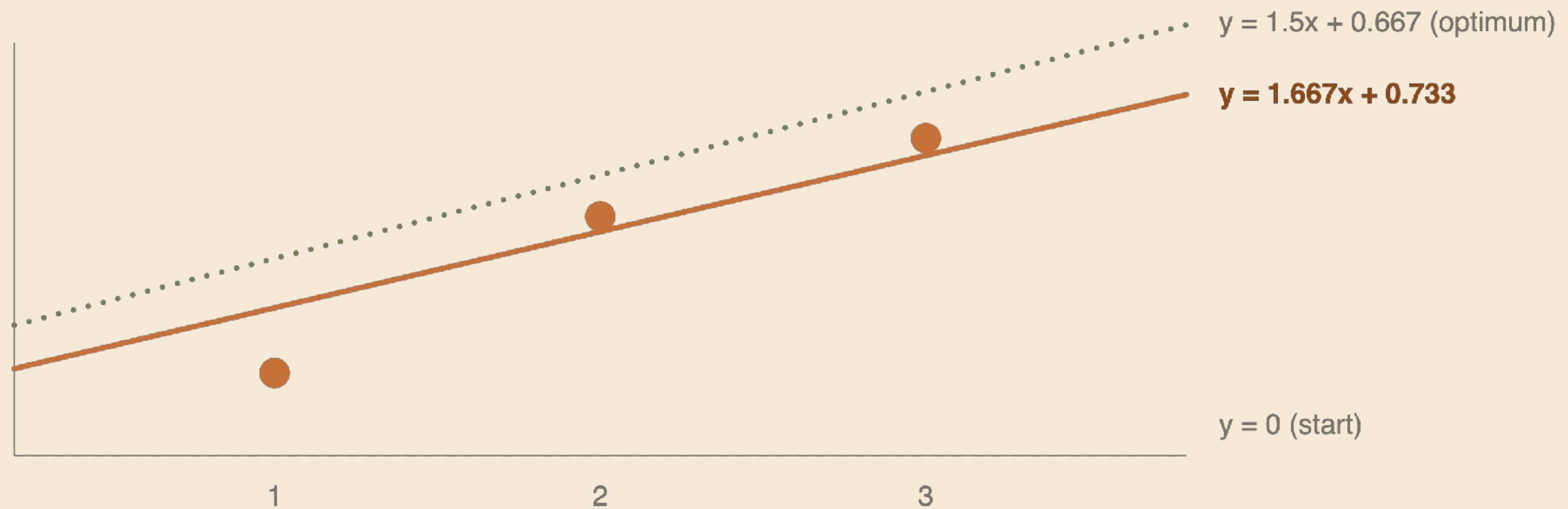
$$\frac{\partial L}{\partial \mathbf{w}} = \frac{2}{3}(-2 - 8 - 15) = -\frac{50}{3} \approx -16.667, \quad \frac{\partial L}{\partial \mathbf{b}} = \frac{2}{3}(-11) = -\frac{22}{3} \approx -7.333$$

One step

$$w \leftarrow 0 - 0.1(-16.667) = 1.66667, \quad b \leftarrow 0 - 0.1(-7.333) = 0.73333$$

ITERATION	W	B	L
0	0	0	15.0000
1	1.66667	0.73333	0.2341
optimum	1.5	0.6667	0.0556

Where it's heading



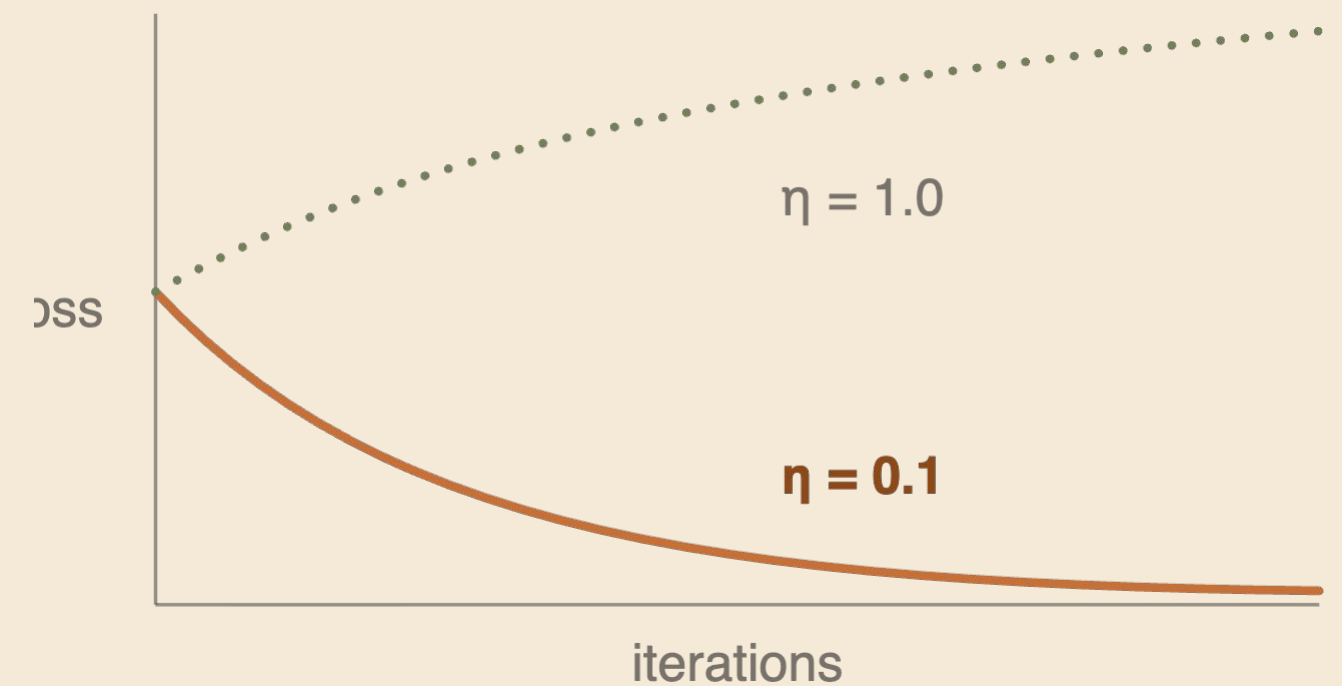
One step covered most of the distance — but not all of it.

Break it: $\eta = 1.0$

Same data, same gradients, one step with $\eta = 1.0$

$\mathbf{w} \leftarrow 16.667, \mathbf{b} \leftarrow 7.333 \rightarrow$ predictions 24.0, 40.7, 57.3
against targets 2, 4, 5

$L : 15 \rightarrow \approx 1522$



Five things to keep

DL is ML with **learned representations** — a subset, not a replacement

A layer is $\phi(Wx + b)$; without ϕ , depth collapses

Supervised learning is four objects: **data, model, loss, optimiser**

The **gradient** says which way to turn every knob; the **chain rule** gets it through a composition

We minimise a **loss**, not a metric, because a loss is differentiable

What today holds up

WEEKS	TOPIC	DEPENDS ON
2–4	Backprop · MLPs & losses · Optimisation	Slides 11–13, 25–30, 34–38
5–7	CNNs · Transfer learning · Evaluation	Slides 21–24, 16–17, 33
9–13	OCR · RNN/LSTM · Attention · Clustering · GANs & VAEs	Slides 11–13, 21–24, 31–32
14–17	Fine-tuning · Endterm · Final project	All of it

“Everything in this course is: pick a function family, pick a loss, follow the negative gradient.”

The next fourteen weeks are increasingly clever choices of those three things.